

项目复盘

1 自我介绍

面试官您好，我是黄思铭，来自北京航空航天大学数学专业，方向是数学与应用数学。我的研究方向是强化学习，同时对推荐系统也有深入研究。技术方面，我熟练掌握Python和PyTorch，也熟悉MATLAB和C语言。我的数学基础比较扎实，在本研阶段，数学分析、高等代数、数理统计和深度学习等理论课程都能获得95左右甚至更高的分数。接下来我将简单介绍一下简历里面提到的三个项目：

第一个项目是“基于元学习的推荐系统冷启动ID生成框架”。推荐系统中新上线的物品由于交互数据不足，导致ID embedding训练不好，传统做法是随机初始化ID embedding或者使用默认值，推荐效果差。我的项目设计了一个冷启动框架，用生成器根据物品的属性特征生成高质量的初始ID embedding。

项目的核心创新是用元学习MAML的方式训练生成器。我把学习每个item的ID embedding当做一个任务，通过学习多个老item的任务，生成器掌握了为item生成好的embedding的能力。

此外还有两个辅助优化：一是，通过随机mask特征槽进行数据增强作为正样本，利用对比学习优化增强生成器的泛化能力。二是用交互用户的平均embedding进行去噪，减轻噪声影响。

在MovieLens-1M数据集上，我的方法将新item的GAUC从提高了1个百分点。

第二个项目是：基于CVaR正则的风险敏感多智能体训练算法。针对多智能体强化学习中的不确定性导致高风险行为的问题，我设计了基于 CVaR 正则 的训练算法，用集成Critic网络构建价值分布，引入条件风险价值惩罚高风险状态。算法在MPE各种环境中碰撞次数下降10%。

第三个项目是卫星对抗环境设计，我设计并实现了一个基于JAX的卫星对抗仿真环境，模拟地球同步轨道上的多卫星对抗场景，环境遵循OpenAI Gym标准接口，可无缝集成主流强化学习算法。

以上就是我的自我介绍全部内容，谢谢！

2 项目介绍：基于元学习的冷启动推荐系统

对于新上线的物品，也就是冷启动item，它的数据不足，导致它的ID embedding训练得很差，进而导致推荐模型推不准。因为ID特征是每个item独有的，所以ID特征只能吃到自己的数据，在自己数据不足的情况下，就更新不好。但其他特征比如类别等于运动，这个特征可以吃到所有运动类item的数据，所以肯定更新得很好。

所以解决冷启动问题的本质，就是解决冷启动item的ID embedding训练不好的问题。传统做法是给新item随机初始化一个ID embedding，但这个embedding质量很差。我的项目就是设计一个冷启动框架，用生成器为新item生成一个良好的初始ID embedding，来替换随机初始化的较差的embedding，以此提高推荐效果。

具体来说，我的项目有三个核心优化点。第一个优化点是构建ID embedding生成器。生成器的输入是item的属性特征比如电影的上映年份标题类型等，输出是一个质量不错的ID embedding。这里的关键是，我用元学习MAML的方式来训练这个生成器。

什么是元学习呢？传统机器学习是一条条样本地训练，而元学习是以任务为单位训练，通过学习多个不同的任务，模型掌握了一种“学习的能力”，当新任务来临时，只需要少量数据就能快速适应。”。我把学习每个item的ID embedding当做一个任务，用多个老item的数据构造多个task，通过对不同task的学习，生成器就学会了如何在少样本情况下，为item生成好的ID embedding这个能力。

然而，在推荐系统中，毫无交互的物品连少量数据都没有，所以传统的元学习是不够的。传统MAML是怎么做的呢？它学习一个好的模型参数初始化，当新任务来临时，用少量样本对模型参数做梯度下降更新，然后在更新后的参数上计算loss，用这个loss反向传播去优化初始化参数。但这有个问题，推荐系统中很多新item是完全冷启动，连少量样本都没有，根本没机会做更新。

所以我做了改进。我不仅用更新后的loss，还同时用更新前的loss，把这两个loss加起来，一起反向传播去更新生成器的参数。并且，我不是更新整个冷启动生成器的参数，而是只更新ID embedding向量。具体训练流程是这样的：先用生成器为训练集的item生成初始ID embedding，在第一个batch数据上计算完全冷启动的loss，这时候还没有更新。然后用这个loss对ID embedding做一步梯度下降，得到更新后的ID embedding，再在第二个batch数据上计算预热阶段的loss。这样生成器在训练时就同时考虑了两个阶段，既能应对0样本的完全冷启动，也能应对有少量数据的预热阶段。当新item来临时，直接用生成器为它生成ID embedding，这个embedding质量就比随机初始化好很多。如果新item积累了少量数据，还可以在这些数据上对ID embedding做一步梯度下降，让它快速适应到新item的真实特征，进一步提升效果。

还有两个优化点相当于是辅助技巧。第一个是基于对比学习的特征空间优化。我把item的属性特征分成两个视图作为数据增强的手段，虽然这两个视图看到的信息不同，但它们描述的是同一部电影，所以应该生成相似的embedding，这是正样本对。同时，我把batch内其他item的视图作为负样本，强制当前item的embedding和其他item的embedding保持距离。通过InfoNCE Loss，拉近正样本对，推远负样本对，这样可以优化embedding空间的几何结构，让相似的item聚在一起，不相似的item分开，增强生成器的泛化能力。

第二个辅助技巧是嵌入去噪。对于冷启动item来说，交互用户数量本来就有限，噪声用户的影响会非常大。我的做法是用冷启动item的最近20个交互用户的平均id embedding来进行去噪。将其拼接t到生成器生成的id embedding上，再过一层MLP网络，可以减轻离群点的影响，从而起到降噪的作用。

训练流程分两个阶段。在讲训练流程之前，要先讲数据集进行处理，以MovieLens-1M电影评分数据集为例，我把交互数大于512的电影当做老item占90%，小于等于512的当做新item占10%作为冷启动

的测试集。而老item中再取90%的数据作为**基础DNN模型的训练集**，剩下10%分为两部分作为**冷启动训练集**。

第一阶段是**预训练推荐模型**，用老item的大量数据训练一个基础的DNN模型，这个阶段就是正常的推荐模型训练。第二阶段是**元学习训练生成器**，冻结第一阶段训练好的DNN参数，只训练生成器的参数。这里用的是MAML的训练方式，每个老item取两个batch的数据，第一个batch模拟**完全冷启动阶段**，用生成器生成的embedding进行预测，计算**冷启动损失**。然后让这个embedding在冷启动损失上梯度下降一步，模拟数据积累后embedding得到更新。再用第二个batch计算**预热损失**，评估更新后的embedding效果。最后用这两个损失加上**对比学习损失**，一起反向传播更新生成器的参数。

预测的时候，当新item来临，直接用生成器根据它的属性特征生成ID embedding，然后用这个embedding参与推荐。如果新item已经有了少量数据，还可以在这些数据上做一步梯度下降，让embedding快速适应到新item的真实特征，进一步提升效果。

实验结果显示，不用冷启动框架的话，新item的**GAUC是0.84**。加入我的冷启动框架后，**GAUC提升到0.85**，提高了1个百分点。

3 为什么强化学习课题组要转投搜广推？

我在强化学习课题组期间，在学习过程中，我发现**推荐系统天然就是一个序列决策问题**：“状态”是系统当前能看到的用户上下文：用户画像、历史交互序列等信息。“动作”就是系统这一步“决定推荐什么、怎么推荐”。奖励就是你优化的业务目标：点击率、停留时长、观看时长、点赞、关注、分享等等。这让我意识到**强化学习和推荐系统有天然的契合点**，激发了我对搜广推领域的兴趣。

强化学习的思维方式对推荐系统很有帮助。首先是**探索利用平衡**，正好对应推荐系统中探索新item和推荐老item。其次是**长期价值优化**，不只看当前点击率，还要考虑用户留存。第三是**序列建模能力**，用户行为是序列的，需要捕捉时序依赖。我做的这个冷启动项目，虽然用的是元学习，但本质上也是在解决如何快速适应新任务的问题，和强化学习中的**few-shot 思想**是相通的。

近年来，我特别关注到**生成式推荐的发展趋势**。传统推荐范式是召回粗排精排重排的多阶段链路，每个阶段独立优化，存在信息损失，依赖大量人工特征工程。而生成式推荐可以**端到端生成推荐结果**，减少由于多阶段流水线拆分而额外引入的性能损失，大模型的涌现能力能更好理解用户意图，还可以**生成个性化的内容描述**，提升用户体验。

我注意到工业界已经在大规模应用RL到推荐系统，比如YouTube用DQN做视频推荐，阿里用虚拟淘宝环境训练推荐策略，字节用Contextual Bandit做冷启动，小红书快手在探索生成式推荐。这让我看到了**强化学习在搜广推领域的巨大应用价值**，也坚定了我转向这个方向的决心。

从就业角度，**搜广推是互联网公司的核心业务**，岗位需求大，技术栈成熟。而且我的强化学习背景是加分项，能带来差异化竞争力。特别是在生成式推荐这个新兴方向，强化学习的探索能力有用武之地。

4 对比学习的正样本构造与语义一致性

我理解面试官的核心疑问是：我把item的属性特征分成两个子集view1和view2，用这两个不完整的特征子集分别生成embedding，然后强制这两个embedding相似，但两个子集信息不同，生成的embedding凭什么要相似，这不是破坏了语义吗？

首先我解释一下对比学习的本质。对比学习的核心就是**数据增强**, 对同一个样本做不同的变换, 得到多个视图, 这些视图表面形式不同, 但**本质语义相同**。然后强制模型学习, 这些不同视图应该映射到相似的表示, 从而让模型忽略表面差异, 抓住本质特征。比如CV领域, 同一张猫的图片可以旋转裁剪变色, 表面像素不同, 但都是猫这个语义。通过对比学习, 模型学会不管角度颜色如何变, 都能识别出是猫。

我的方法本质上和标准对比学习的数据增强是一样的, 只是**增强方式不同**。CV可以用旋转裁剪, NLP可以用回译同义词替换, 但推荐系统的特征是离散的, 不能像图像那样连续变换。我采用的数据增强方式是**随机mask掉一部分特征槽**, 这样同一新item的两个视图看到的信息不同, 但它们描述的是**同一部电影**, 这就是数据增强。

但这里有个问题, 如果直接把某些特征mask成0, 可能会完全改变电影的属性表示, 相当于变成了另一部电影, 破坏了语义。为了解决这个问题, 我加入了一个SENet风格的注意力机制**W-SENET**。它的作用是, 当某些特征被mask成0之后, 模型可以**自适应地调整剩余特征的权重**, 给重要的特征更高的权重, 从而补偿被mask掉的信息。这样即使mask掉了一部分特征, 生成的embedding也不会完全偏离原来的语义, 而是保持在合理的范围内。

通过对比学习, 我强制生成器学习, 不管从哪个角度看, 也就是不管用view1还是view2, 都应该生成相似的ID embedding。这和CV中强制不同角度的图片生成相似的表示是一个道理。这样做的好处是什么呢? 生成器学会了从**不完整信息中提取本质特征**。当新item来临时, 即使某些特征缺失, 生成器也能生成**合理的embedding**, 这就提升了鲁棒性和泛化能力。

总结一下, 我的方法和标准对比学习的数据增强本质是一样的, 都是对同一个样本做不同的变换, 然后强制不同变换生成相似的表示。只是CV用旋转裁剪, 我用特征子集划分, 但**核心思想是一致的**。这不是破坏语义, 而是让模型学习**语义的不变性**, 从而提升泛化能力。

5 冷启动是user-item, 为什么你做item-item?

冷启动问题确实是预测**user-item交互**, 但解决方案可以从不同角度切入。传统的**user-item建模思路**是协同过滤, 输入user id和item id, 输出预测评分或点击概率, 但问题是**新item没有user交互历史**, 协同过滤失效。而**item-item建模**的思路是内容增强, 核心是找相似老item迁移它们的知识, 输出新item的**良好ID embedding**, 然后用这个embedding参与user-item预测。

我的方案是**item-item是手段, user-item是目标**。完整流程分两个阶段。阶段一是**item-item相似度**, 也就是生成器训练, 新item找到K个相似老item, 生成新item的ID embedding。阶段二是**user-item预测**, 也就是推荐模型, 把user特征、新item的生成embedding、其他特征输入DNN, 预测点击率。

为什么强调item-item呢? 第一, **冷启动的本质是item端问题**。新item没数据, user是有数据的, 问题出在item的ID embedding训练不足, 所以要从item端解决。第二, **item-item是手段, user-item是目标**。通过item相似性生成embedding是手段, 最终还是预测user对item的兴趣是目标。第三, **元学习的任务定义**, 每个task是学习一个item的embedding, 不是学习一个user的偏好, 因为item是冷启动的, user不是。

总结一下, 不是只有item-item, 而是**item-item相似生成item embedding, 然后user-item预测**, 用item-item解决user-item问题。如果是user冷启动, 思路会反过来, **user-user建模**找相似老用户生成新用户的embedding, 然后用这个user embedding参与user-item预测。但我的项目聚焦**item冷启动**, 所以是item-item建模。

6 为什么要用第一段Loss，既然完全冷启动，那应该没有用户也就没有label？

第一段Loss用的不是真正0数据的冷启动item，而是用老item的历史数据来模拟冷启动场景。具体来说，我用生成器为老item生成一个embedding，替换掉它预训练好的embedding，然后在它的一部分历史交互数据上计算loss。这模拟的是：如果这个item是新的，只能用生成器生成embedding，效果如何。通过优化这个loss，生成器学会了即使没有预训练，也要生成高质量的初始embedding。这样当真正的新item来临时，生成器就能为它生成一个好的起点。

第二段Loss模拟的是预热阶段。在第二段Loss的基础上，我让生成的embedding在第一批数据上做一步梯度下降，得到更新后的embedding。然后在老item的另一部分历史交互数据上计算loss。这模拟的是：当新item积累了一些数据后，embedding经过更新，效果能提升多少。通过优化这个loss，生成器学会了不仅要生成好的初始embedding，还要生成容易适应的embedding，也就是说，这个初始embedding在少量数据上更新后，能快速达到好的效果。这样当真正的新item积累了少量数据时，就能快速适应到最优状态